LABSHEET 6: BACK-END INFRASTRUCTURE

Demonstrating execution path mappings, VS Code configuration settings, and automated deployment diagnostic scripts for Python and Django environments.

TASK 6.1: VIRTUAL ENVIRONMENT & PROJECT SETUP

Initialize an isolated virtual execution ecosystem using venv and scaffold a Django core application layout.

Execution Commands (Terminal / PowerShell):

```
# Create isolated virtual environment using hidden directory
python -m venv .venv

# Activate virtual environment
.\.venv\Scripts\Activate.ps1

# Upgrade package manager and install Django framework
python -m pip install --upgrade pip
pip install django

# Export environment requirements
pip freeze > requirements.txt

# Initialize project within the workspace root directory
django-admin startproject core_app .
python manage.py runserver 8000
```

Summary of Changes:

- Adopted standard `.venv` naming convention for environment isolation.
- Upgraded `pip` prior to framework installation to prevent package warning logs.
- Initialized project using root path binding to streamline project structure.

TASK 6.2: VS CODE PEP8 AUTOMATION

Map custom user shortcuts and syntax formatting workflows inside VS Code configuration files for Python PEP8 compliance.

Formatter Installation:

```
pip install autopep8
```

Configuration File (.vscode/settings.json):

```
{
  "python.analysis.typeCheckingMode": "basic",
  "[python]": {
    "editor.defaultFormatter": "ms-python.autopep8",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.organizeImports": "explicit"
    }
  }
}
```

Summary of Changes:

- Applied formatting settings within a language-specific block (`[python]`).
- Configured automatic import organization alongside format-on-save triggers.
- Utilized `autopep8` as the dedicated PEP8 formatter engine.

TASK 6.3: AUTOMATED DEPLOYMENT DIAGNOSTIC SCRIPT

Script an automated deployment check file that validates systemic runtime pathways, framework dependencies, and global server keys.

Diagnostic Verification Script (check_env.py):

```
import os
import sys
import django

def run_environment_check():
    # Validate environment runtime pathways and active settings module
    settings_module = os.getenv("DJANGO_SETTINGS_MODULE")

    border = "=" * 48
    print(border)
    print("        SYSTEM DEPLOYMENT DIAGNOSTICS        ")
    print(border)
    print(f"{'Python Version':<22}: {sys.version.split()[0]}")
    print(f"{'Django Version':<22}: {django.get_version()}")
    print(f"{'Interpreter Path':<22}: {sys.executable}")

    status = f"YES ({settings_module})" if settings_module else "NO (Missing)"
    print(f"{'Settings Configured':<22}: {status}")
    print(border)
    print("Validation check finished successfully.")

if __name__ == "__main__":
    run_environment_check()
```

Sample Console Output:

```
================================================
        SYSTEM DEPLOYMENT DIAGNOSTICS
================================================
Python Version        : 3.11.4
Django Version        : 5.0.2
Interpreter Path      : C:\dev\.venv\Scripts\python.exe
Settings Configured   : YES (core_app.settings)
================================================
Validation check finished successfully.
```

Summary of Changes:

- Encapsulated execution logic inside a structured module function.
- Used Python string formatting specifiers for aligned tabular output.
- Provided dynamic reporting for the active Django settings module key.